

JVMud User Manual

JVMud Project

Version 0.1.0-SNAPSHOT

Table of Contents

How to use this manual	1
Getting Oriented	2
Installing And Running	6
Playing And Testing	13
Operating JVMud	14
LPC Authoring	16
Mudlibs	39
Developer Reference	40
Troubleshooting	42
Appendixes	45

How to use this manual

JVMud is an LPC/LPMud engine for the JVM. This manual explains how to understand its design, run a local world, inspect LPC objects, and develop mudlib content against the current repository.

The documented version is 0.1.0-SNAPSHOT. The platform provides a compiler, world model, hosted instances, Telnet sessions, storage adapters, and an admin shell. Compatibility is incremental; a successful build or preload scan does not prove complete gameplay or production readiness.

- **Evaluating JVMud:** read *Getting Oriented* for philosophy, architecture, and the core package map.
- **Running or playing a local world:** follow *Installing And Running* and the *Small Mercies walkthrough*, then *Playing And Testing*. Commands in terminal blocks run from the repository root.
- **Operating or debugging:** use *Operating JVMud* and *Troubleshooting*.
- **Writing LPC:** follow *LPC Authoring* and *Mudlibs*; keep the configuration and command references nearby.
- **Changing Java:** start with the package map, then *Developer Reference*.

Examples identify whether input belongs in a terminal, the admin shell, or a Telnet client. Replace angle-bracket placeholders with actual values; do not type the angle brackets. A mudlib is the LPC source and configuration that define a game.

The manual owns procedures, concepts, and LPC reference material. Java API pages describe Java contracts. The website provides project information and links into these two documentation sources.

Terms used in procedures

A **Player** is an identity; a **Session** is a connection; a **Persona** is the Entity through which that Player acts in a World. A **Place** provides locality, a **Link** connects Places, and a **Location** identifies containment. A loaded LPC object can also be a service with no physical presence.

A **manifest** is the profile's `.config` file. It selects source paths, startup objects, lifecycle mappings, language features, and engine capabilities. A **profile** includes this manifest and the mudlib-side adapters needed to host that content. Legacy terms such as `room` and `heart_beat` are used when naming actual LPC paths or compatibility methods.

Getting Oriented

Start with what JVMud means by a world, then follow how its packages collaborate. The architecture is an ownership model; it is not a claim that every planned engine behavior is complete.

Overview

JVMud compiles LPC source to JVM bytecode and loads that code as live objects. The engine models shared world concepts; a mudlib supplies fiction, commands, rules, and presentation. The hosted instance brings these parts together, while Telnet gives players a line-oriented way to participate.

The distribution includes Small Mercies, a tiny teaching world. Start with its walkthrough, or choose or author a compatible LPC source tree and declare its hosting requirements in a manifest. Shared launchers always require that manifest explicitly.

The Java code is one Maven project with responsibility-based packages. The packages described here are not separately released Maven modules.

The JVMud Philosophy Of MUD

JVMud is built around eight core concepts:

- Game
- Text
- Interactive
- Multiplayer
- World
- Persistence
- Temporality
- Presence

Together, these concepts define what JVMud means by MUD.

A MUD is an interactive, multiplayer, text-only game set in a persistent, temporal, player-present world.

JVMud is not a general-purpose simulation framework, chat server, collaboration platform, generic game engine, or generic MUD framework. It is specifically and by design an LPMud engine: a platform for imagining, creating, operating, and playing LPC-authored game worlds in the tradition of LP MUDs.

Game

The world exists to be played. JVMud assumes rules, mechanics, goals, challenges, progression, or other game-like structures.

Text

Text is the primary medium through which the world is perceived and manipulated. It is a first-class part of the engine's design, not a skin over a visual game.

Interactive

Players can affect the world, and the world can affect players. Actions have consequences.

Multiplayer

Multiple players inhabit the same world simultaneously. The world is shared.

World

The game takes place within a concrete virtual domain. JVMud describes this domain in terms of Places, Links, Entities, and Movement.

Places give the world locality. Links make Places traversable. Entities are the things that exist in that structure. Movement changes where an Entity is present.

Persistence

The world and its state endure independently of any individual player's session. A player may disconnect and later return to a world whose state has continued to exist.

Persistence does not require eternal continuity. Reboots, resets, and selected save/restore behavior can all be compatible with a persistent world.

Temporality

The world can change through time. Events occur, actions have consequences, and state evolves.

Temporality is separate from Persistence: a world can have active time, saved state, restored state, and reset state without treating all of those as the same idea.

Presence

Presence means that a player experiences the game from within the World. The player is not an outside observer manipulating a simulation from above.

In JVMud terms, a player participates through a Persona: an Entity that serves as the player's current point of perception and action. A Persona is located somewhere, and that location determines what the player can perceive and what actions are locally available.

Players are somewhere, not everywhere. Presence creates locality of information, which makes exploration, discovery, concealment, surprise, navigation, and mystery possible.

LPC Compatibility And Engine Vocabulary

JVMud has one mudlib and language target: LPC for LPMud-style worlds. Its compatibility work exists to make JVMud a deeper LPMud platform, not to broaden it into a multi-language MUD framework.

An LPMud, in JVMud's sense, is a game world authored in LPC, compiled into live game objects, and able to be rewritten, recompiled, and reloaded while the game continues running.

That does not mean JVMud adopts every legacy LPMud driver term as engine ontology.

Legacy names such as rooms, objects, heartbeats, applies, call_outs, and master objects may appear in compatibility surfaces, mudlib source, and LPC-facing documentation. The engine-facing model remains JVMud's own model: Game, Text, Interactive, Multiplayer, World, Persistence, Temporality, and Presence, built from Places, Links, Entities, Players, Sessions, and Personas.

Architecture

JVMud has four top-level Java packages: `language`, `execution`, `communication`, and `storage`.

`language` turns LPC source into executable JVM code and supplies the runtime support that code needs. It owns preprocessing, scanning, parsing, semantic analysis, IR, bytecode generation, efun contracts, loading, and diagnostics.

`execution` separates three responsibilities:

- `application` starts JVMud, owns public endpoints and administrator authority, supervises sandboxed worker JVMs, and supports installation updates. It remains available with zero mudlibs.
- `model` defines worlds, Places, Entities, Locations, Links, identities, time, and mudlib boundary contracts. `WorldRuntime` enforces rules such as single containment and valid movement. These concepts do not own process supervision.
- `instance` assembles language runtime facilities and model state into one running mudlib. It boots content, attaches Personas, invokes lifecycle hooks, and serializes input and ticks on the worker's execution queue.

`communication` owns human-facing interaction. Its `transport` packages handle sockets, Telnet, administration framing, and connection lifecycle. Its `admin` package interprets administrative commands and maintains session state; its `console` package supplies the terminal client. Structured client messages and text presentation live in `protocol` and `output`. Player commands continue to be interpreted by the `mudlib`.

`storage` owns durable representations and adapters, including filesystem accounts, LPC object state, JDBC services, and administrator identities and grants. Execution uses these services without owning their storage formats.

The `mudlib` supplies authored content, fiction, object behavior, player commands, and compatibility policy. It lives under `mudlibs/`, outside the Java package tree. A player connects through communication code to an execution instance, which runs language-compiled behavior against model state and storage services.

These packages organize responsibilities within the project. The worker process sandbox, rather than a Java package name, enforces isolation between `mudlibs` and the supervising application.

Core packages and repository tour

All Java package names below begin with `io.github.protasm.jvmud`.

Package	Responsibility and starting points
<code>language</code>	LPC compilation and runtime support. Start with <code>JVMudCompiler</code> , <code>pipeline</code> , and <code>exec.LPCRuntime</code> . <code>runtime</code> supports generated bytecode; <code>diagnostics</code> contains standalone compilation checks.
<code>execution</code>	<code>application</code> owns startup, worker supervision, and updates; <code>model</code> defines worlds, identities, time, and <code>mudlib</code> contracts; <code>instance</code> assembles and runs one <code>mudlib</code> .
<code>communication</code>	<code>transport</code> owns connection mechanics, <code>admin</code> interprets administrative commands, and <code>console</code> supplies the terminal client. <code>protocol</code> and <code>output</code> handle structured messages and text presentation.
<code>storage</code>	Durable adapters under <code>filesystem</code> , <code>jdbc</code> , and <code>admin</code> .

`src/main/java/` contains these packages; `src/test/java/` mirrors them. `mudlibs/<name>/source/` contains LPC content, and `mudlibs/<name>/jvmud/` contains manifests, adapters, and profile-specific scripts. Inspect the selected profile before assuming all `mudlibs` have the same storage layout.

`manual/` contains editable AsciiDoc; `docs/` contains the site, generated manual, and API. Shared tools live in `scripts/`. `pom.xml` defines dependencies and build profiles. See [Documentation Generation](#) before editing generated pages.

Installing And Running

This chapter gets one local listener running and proves that a player can reach it. Database requirements depend on the selected profile.

Installing

Installing a preview package

Download the archive matching your platform from [Downloads](#). A POSIX terminal and a Telnet-capable MUD client are required. The platform packages include Java; the runtime-free package requires Java 21 or newer. Maven is not needed.

In a terminal, go to the writable folder containing your download. For Apple Silicon, run the commands below; for another platform, replace the archive filename with your download:

```
mkdir current &&  
tar -xzf jvmud-0.1.0-preview.7-macos-aarch64.tar.gz --strip-components=1 -C current &&  
cd current &&  
scripts/jvmud-start smallmercies
```

Keep the installation in `current` across releases. These commands stop if that directory already exists; use the updater below for an existing installation.

Wait for the mudlib's `state=RUNNING` status, then connect a Telnet-capable MUD client to `127.0.0.1:4000` with SSL/TLS disabled. Choose a guest name, gender, and class. Try `help`, `look`, and `north`. Stop the server with `Ctrl+C`.

Update an installation

From your existing `current` directory:

```
scripts/jvmud-update --check  
scripts/jvmud-update
```

The updater verifies the latest matching download, stops this installation's servers cleanly, and creates a full backup in `../backup/`, beside the distribution folder. It updates JVMud and shipped mudlib `jvmud/` files, then restarts the servers. Other mudlib content and saved data remain in place. Starting with `preview.6`, comment-only configuration differences do not block updates, edited Markdown documentation is backed up and replaced, and genuine adapter/configuration conflicts are reported together before downtime. The updater already installed performs each upgrade. External files and databases need separate backups.

The installation folder keeps its name across releases; the updater reports the installed version. Do not rename a running installation. Stop its servers cleanly first, then relaunch them from the new location with their original options and update any external paths. The bundled Java runtime is available through `jre`; server logs live in each mudlib's `jvmud/log/` directory. Players must reconnect after an update. Earlier previews without the updater require a manual migration; see the package README for recovery and migration details.

Building from source

You need a JDK that can compile Java 21, Maven, Git, and a POSIX shell for the repository launch scripts. Some profile-owned smoke scripts also require Bash or Python 3. A Telnet-capable client is needed for interactive play. The bundled Small Mercies walkthrough needs no database or saved-account setup.

```
git clone https://github.com/protasm/jvmud.git
cd jvmud
java -version
mvn -version
mvn -DskipTests compile
```

Check that Maven reports the intended JDK. The compiler release is set to 21 in `pom.xml`. The first build needs access to Maven dependencies; later builds may use the local cache. Successful compilation creates `target/classes/`.

For an existing checkout, run commands from its root. Do not clone over local work. Tests are described in [Testing And Verification](#); compiling alone does not run them.

Running JVMud

For a ready-to-run example, use [First steps with Small Mercies](#). To host your own content, select a JVMud-compatible LPC source tree and its manifest. JVMud is the host; it does not choose game content implicitly. The manifest must describe real source files, including an initial Place and the profile's player behavior. Small Mercies is the bundled teaching example; the same host contract applies to other compatible content.

In a terminal at the repository root:

```
scripts/jvmud-start <mudlib-config-file>
```

Replace `<mudlib-config-file>` with the absolute path to your manifest, or a path relative to the repository root. Do not type the angle brackets. The script compiles Java, resolves the dependency classpath, and runs the Telnet server in the foreground. Wait for the preload summary followed by the engine endpoint report and the mudlib's `state=RUNNING` status before connecting.

The launcher defaults to loopback port 4000. Use `--bind 0.0.0.0 --port 4000` to accept incoming

connections on all IPv4 interfaces, or choose a specific local address and a different port. Clients use the server's actual address; allow the selected TCP port through applicable firewalls and router forwarding. Each listener needs an available address/port pair. Mounted profiles, if declared, must also be present and valid. See [Configuration Reference](#) for manifest keys.

To capture detailed startup failures:

```
scripts/jvmud-start --trace-startup-loads <mudlib-config-file> > target/startup.log  
2>&1
```

This still runs in the foreground. Inspect the log from another terminal and stop with Ctrl+C. Trace output can be large; begin with the preload summary and first failure. To see launch syntax, use `scripts/jvmud-start --help`.

Ctrl+C stops the engine and its worker JVMs. Persistence remains mudlib-specific; shutdown does not promise a snapshot of every loaded object. Engine administration is always available on its separate TLS port (default 4001) and local Unix socket. Use the administration console workflow described in the next section.

Offer several mudlibs

```
scripts/jvmud-start smallmercies lp245
```

The engine player port presents a selection menu, including for a single mudlib. Each mudlib receives a direct player port and a separate TLS admin port, shown in engine status. Each worker owns its execution queue and configurable clock. Quitting a mudlib closes the connection; reconnect to select another.

Starting with no manifests is valid. Use engine administration to start mudlibs later and to request explicit player/admin port pairs. A failed mudlib startup leaves engine administration and other mudlibs available.

First steps with Small Mercies

Small Mercies is JVMud's bundled toy mudlib: five rooms, a few talkative NPCs, and sparring opponents with more confidence than good sense. It demonstrates a native LPC profile without database setup, account registration, or saved characters. Use it to learn the player flow and to find small source examples.

Launch and create a guest

From the repository root:

```
scripts/jvmud-start mudlibs/smallmercies/jvmud/smallmercies.config
```

Wait for the engine endpoint report and the mudlib's state=RUNNING status. In a second terminal or Telnet-capable MUD client, connect to 127.0.0.1 on port 4000:

```
telnet 127.0.0.1 4000
```

Answer each prompt separately:

1. Choose a name containing 2–16 letters. It must be different from the names of players currently connected.
2. Choose male or female.
3. Choose warrior or mage.

You arrive in the **Village Square**. Type `help`, then `score`. There is no password. A guest character exists for one connection: disconnecting ends that character's visit, and reconnecting starts character creation again.

Class	STR	INT	DEX	Starting HP
Warrior	12	6	9	32
Mage	6	12	9	26

Warriors use STR for their attacks; mages use INT for sparks. DEX helps dodge. `score` also shows victories and the current opponent, when fighting. Gender changes the character description, not the attributes or rules.

Explore the whole world

Every exit has a return route. Read `look` to see a description, exits, and nearby characters. Use full directions, their initials, or go `<direction>`.

Place	Exits
Village Square	west to The Resting Hero; north to the Municipal Herb Garden; east to Very Short Lane.
The Resting Hero	east to the square.
Municipal Herb Garden	south to the square.
Very Short Lane	west to the square; down to the Cellar of Mild Peril.
Cellar of Mild Peril	up to the lane.

From a fresh arrival in the square, enter these player commands one at a time:

```
look
```

```
talk mayor
west
talk innkeeper
rest
east
north
look goose
south
east
down
look rat
up
west
```

You finish back in the square. `rest` restores full health only at the inn. `talk` addresses an NPC in your current Place. Try `who` to list connected adventurers. `say Hello!` and `smile`, `wave`, `bow`, or `laugh` provide local communication. Connect a second guest to try these with another player.

Try a short sparring match

From the square, go `north`, then type `attack goose`. Combat repeats every two seconds; you do not need to repeat the attack command. `kill goose` is an alias. Use `stop` or move `south` to end the bout. The rat in the cellar is the other sparring opponent; the mayor and innkeeper are friendly.

A defeated NPC rests for 20 seconds before recovering. A player defeat sends you to the inn with 1 HP; `rest` restores full health. NPCs spar with one challenger at a time; their remaining HP persists when a visitor stops or leaves. There is no player-versus-player combat. If an opponent is resting or occupied, return later rather than repeatedly issuing `attack`.

Use `score` to check health and victories, then `quit` to end your visit. `//quit` is the default transport disconnect control. Neither creates a saved character. NPC state remains shared while the server runs; restarting resets the world. Stop the server with `Ctrl+C` in its terminal when finished.

Read the implementation

The manifest selects `room/square` and `player/adventurer`, maps object loading to `create`, interaction scope to `init`, session connection to `connect`, and session disconnection to `disconnect`. It grants `session_control` and maps recurring ticks to `tick`; there is no filesystem-account policy or database.

Path under <code>mudlibs/smallmercies/</code>	What to learn
<code>jvmud/smallmercies.config</code>	Explicit source root, preload manifest path, Persona, initial Place, lifecycle mappings, and capability grant.
<code>source/preload_objects</code>	Startup object paths, one per line in load order.
<code>source/lib/room.c</code>	Shared descriptions, authored exit lookup, and NPC creation.

Path under <code>mudlibs/smallmercies/</code>	What to learn
<code>source/room/*.c</code>	Small Place definitions with prose, exits, and resident NPCs.
<code>source/player/adventurer.c</code>	Guest creation, command registration, movement, communication, score, and recurring combat.
<code>source/npc/villager.c</code>	NPC identity, dialogue, single-challenger combat, and deferred recovery.

These LPC examples use JVMud-native efuns. Game rules stay in the mudlib; the host supplies execution, session, location, and scheduling services. Read the source before changing behavior, then replay the relevant route after restarting. Use a separate copy for experiments you do not intend to keep in the distribution.

The two-guest live Telnet check lives at `src/test/scripts/smallmercies/smoke-play.py`.

Connecting With Telnet

In a second terminal, use a Telnet client:

```
telnet 127.0.0.1 4000
```

If your system has no `telnet` command, use a Telnet-capable MUD client with host `127.0.0.1` and port `4000`. Follow the mudlib's login prompts, then try `help` and `look`. Use `//quit` to disconnect from profiles using the default transport prefix. The listener keeps running until stopped in its own terminal.

Optional GMCP integration

JVMud's Telnet listener supports ordinary line-oriented clients and negotiates the Generic Mud Communication Protocol (GMCP) as Telnet option 201. The server offers GMCP with `IAC WILL GMCP`; clients that accept it can exchange UTF-8 GMCP subnegotiations without affecting the visible text stream.

Mudlibs with the `session_control` engine capability can use:

- `jvmud_gmcp_enabled()` or `jvmud_gmcp_enabled(user)` to test negotiation state.
- `jvmud_send_gmcp(package)` for a package without a payload.
- `jvmud_send_gmcp(package, payload)` to JSON-encode LPC strings, numbers, arrays, and string-keyed mappings.

An interactive LPC object may implement `client_protocol_changed(string protocol, int enabled)` to react to negotiation and `receive_gmcp(string package, mixed payload)` to receive decoded client messages. JVMud owns Telnet framing and JSON conversion; mudlib code owns the meaning of packages such as `Room.Info` and `Char.Vitals`.

Smoke Tests

For the bundled toy, run its focused integration test from the repository root:

```
mvn -Dtest=SmallMerciesTest test
```

This boots the real profile and exercises character creation, movement, room-local communication, and scheduled combat. It does not start a TCP listener; use the route in [First steps with Small Mercies](#) for live transport verification.

For an automated live check, start a fresh Small Mercies listener as described in the walkthrough. Keep it running, then use a second terminal:

```
python3 src/test/scripts/smallmercies/smoke-play.py
```

The script connects to loopback port 4000, creates two disposable guests, and checks login, both classes, chat, an emote, the tutorial route, combat, score, rest, and quit. Success prints `PASS: live guest login...`. It does not launch or stop the server. Use a dedicated development listener so another player's sparring match or a resting NPC does not interfere.

To compile every object in Small Mercies' `source/preload_objects` manifest:

```
scripts/scan-mudlib-preload-compile.sh mudlibs/smallmercies/jvmud/smallmercies.config
```

The scanner checks compilation; the focused test above also exercises gameplay.

A host smoke test should prove the complete route from launch to interaction, using the profile you intend to run:

1. Start the listener with its explicit manifest and wait for the listening message.
2. Connect a Telnet client and complete the configured login or entry flow.
3. Exercise a known command and verify its expected text.
4. Move through a known Link and verify the new Place.
5. Disconnect and reconnect to check cleanup and any state the profile promises to save.
6. Stop the listener and retain the first error if a step fails.

Use a disposable account and state directory selected for the test. Profile-owned scripts may automate these steps, but the shared engine does not prescribe game names, account names, commands, locations, or credentials.

Passing proves that profile's particular transcript. It does not prove all language behavior or multi-user reliability. Separate compiler assertions, preload scans, and live player evidence when reporting results.

Playing And Testing

Player commands belong to the selected mudlib. JVMud handles transport controls and delivers ordinary input to the hosted world.

Login Flow

Connect only after the listener reports that it is ready. Follow the profile's prompts rather than sending gameplay commands before entry is complete.

There are two configuration patterns:

- With `session_policy = filesystem_accounts`, JVMud performs local account-file and password-verification work. It then dispatches `player_persona_resolved` and `player_entered_world` to mudlib-owned methods selected in the manifest.
- Without a managed session policy, the configured interactive object supplies the login conversation through its mapped session-connection lifecycle method.

`persona_object` selects the interactive LPC source; `player_object` is the legacy manifest spelling. Method names and prompts belong to that source tree, not to a universal JVMud game flow.

For a repeatable test, record the manifest, whether the account is new or existing, the prompt where input was sent, and the first unexpected reply. Exclude passwords and private account fields from shared transcripts.

Basic Commands

Read the selected world's welcome text and help after entry. A profile may provide verbs such as `help`, `look`, or movement directions, but JVMud does not require a universal gameplay vocabulary. Use the exact verbs and exits documented by your authored content.

For a first route, inspect your current Place, choose one listed exit, move, and verify that the description changes. Record the expected text in your test. These are player-client inputs; the local admin shell has a separate grammar.

With the default transport prefix, `//help` shows connection controls and `//quit` disconnects the client. A mudlib's own exit command can run game-specific saving and cleanup. For persistence tests, exercise that command as well as transport disconnect/reconnect.

Command Routing

Input passes from the Telnet session to the hosted instance and the profile's LPC command behavior. The configured `transport.control_prefix` reserves an escaped namespace for connection controls; its default is `//`.

- `//help` and `//quit` are transport controls in profiles using the default transport prefix.
- `look` and movement verbs go to the mudlib after login.
- Ordinary single-slash commands remain available to mudlib code.
- `load`, `inspect`, and `reload` run through the authenticated admin connection against the selected live mudlib. They are not player Telnet commands.

If a command is unrecognized, first identify which of these contexts received it. Then check the active profile's lifecycle mappings and command registration.

Transcript Regressions

For a useful regression report, record the repository revision, manifest path, startup summary, client input, expected result, actual output, and server error. Use a small route with known starting state. A successful login is a different assertion from successful movement, persistence, or selection from the mudlib menu.

Start with the Small Mercies walkthrough and the relevant tests under `src/test/java/io/github/protasm/jvmud/execution/instance/` and `transport/`. Add a focused assertion for the first wrong behavior, then replay the route against a restarted listener when runtime code changes. Avoid treating a long transcript with no assertions as proof of compatibility.

Operating JVMud

Use the local admin shell to investigate object behavior, the manifest to define hosting policy, and the server log to diagnose a running listener.

Administration consoles

The engine starts independently of mudlibs. Its player port presents a public menu; its TLS admin port manages mudlib lifecycles and engine-owned administrator grant records. Each mudlib runs in a separate worker JVM with its own player/admin port pair. Mudlib administration operates on that worker's live objects.

Bootstrap locally as the engine's OS account:

```
scripts/jvmud-start
# In another terminal:
scripts/jvmud-console --socket ~/.jvmud/engine-4000/engine.sock
```

```
admin-create operator
grant operator engine
start /absolute/path/to/mudlib/jvmud/mudlib.config 4100 4101
```

Store the issued random token securely; it is displayed only once. The engine stores its hash and grants. Remote connections require the certificate fingerprint printed by the engine, obtained through a trusted channel:

```
scripts/jvmud-console --host server.example --port 4001 --user operator --fingerprint <SHA-256>
```

The console prompts for the token. Use `--token-file <private-file>` when no interactive terminal is available. Engine grants include all mudlibs; `grant <name> mudlib:<id>` limits an administrator to one mudlib. Grants are independent of OS accounts, player accounts and in-game wizard privileges.

Connect to a mudlib's admin port for objects, inspect, call, reload, and other mudlib commands. quit disconnects only the console. stop and restart are engine commands; shutdown stops the engine and all workers. A worker crash closes its connections while leaving the engine and other mudlibs running.

The private engine state defaults to `~/jvmud/engine-<player-port>` and contains its recovery socket, registry, TLS identity and bounded worker logs. Override it with `--state-dir`. Keep it outside mudlib trees. macOS requires `sandbox-exec`; Linux requires bubblewrap and user namespaces. Sandbox startup fails closed.

See the repository's `docs/ENGINE-ADMINISTRATION.md` for isolation limits, timeout behavior, certificate management and the complete operating workflow.

Runtime Model

`WorldRuntime` enforces single containment and rejects containment cycles. Places connect through Links; Entities can be contained by Places or Entities. Players participate through Personas attached to Sessions. Time-dependent work uses the engine scheduler.

Hosted LPC execution also keeps object environment and inventory state in its runtime context. Do not assume every loaded service object is a world-present Entity, or that all compatibility bookkeeping has been unified with `WorldRuntime`. The repository's `docs/DEFERRED_WORK.md` records this boundary and the remaining world-event/concurrency work.

The `filesystem_accounts` policy stores account records under `accounts/` in its selected mudlib root. LPC object save/restore and JDBC data are separate storage surfaces. Before an upgrade or reset, identify the active profile's state paths and database, stop writers, and preserve those stores. The project does not yet provide a universal backup/restore command or a verified production recovery procedure; test restoration on a disposable copy of the profile's data.

Mudlib Boundary

The manifest declares how a profile participates in JVMud. `initial_place` selects the starting Place. `persona_object` (or `legacy_player_object`) selects the interactive LPC object. Preloads, include paths, lifecycle methods, syntax features, and capabilities are explicit.

To adapt a profile:

1. Begin with its current `.config` file and identify `mudlib_root`.
2. Verify the initial Place, Persona, and preload paths exist.
3. Map JVMud lifecycle events to the mudlib's method names.
4. Enable only the syntax and engine capabilities the profile needs.
5. Launch it, inspect the preload summary, and test login and a small command route.

Unknown keys fail validation. `language_features` controls optional syntax; `engine_capabilities` authorizes capability-gated efuns. Neither is a substitute for the other. See [Configuration Reference](#) for the supported keys.

`docs/ENGINE_MUDLIB_CONTRACT.md` is the detailed design contract. Keep native world semantics in the engine and profile vocabulary in manifests or dedicated shims.

Example declaration

The following is a manifest shape, not a complete runnable world. Each source path must exist in your authored tree; the chosen method names must match it.

```
game_id = <your-world-id>
mudlib_root = ../source
initial_place = place/start
persona_object = persona/player
lifecycle.player_session_connected = connect
transport.control_prefix = //
```

Add preloads, mapped world-entry behavior, account policy, syntax gates, and host capabilities only as required by that content. Validate startup before testing player interaction.

LPC Authoring

The [Small Mercies source map](#) links runnable examples of rooms, commands, NPCs, and scheduling. Start with a typed object and test it through the console attached to a development server. Then connect its behavior to the selected mudlib's commands and lifecycle. The language overview below preserves practical LPC examples; compatibility names such as `write` and `sizeof` depend on the active profile's mappings.

Supported LPC

JVMud supports LPC as the source language for LPMud-style game worlds. The goal is not to preserve every historical driver quirk. The goal is to provide a clear, practical LPC platform for writing mudlib code that can be compiled, loaded, inspected, reloaded, and hosted by JVMud.

There is no single authoritative LPC. LPC grew through the work of hobbyists, enthusiasts, and mudlib authors building real worlds on real drivers. Over time, different drivers and mudlibs carried the language in different directions. Many LPC features are therefore best understood as dialect features, compatibility conventions, or driver-specific behavior rather than as parts of one universal standard.

When this manual says JVMud supports LPC, it means JVMud supports the LPC language surface described here, plus the compatibility behavior needed to host the mudlibs JVMud is designed to run.

This topic describes the supported language surface at a high level. It is a working authoring guide, not a complete grammar reference.

Source Files And Objects

An LPC source file compiles to a loadable game object. Source files may define:

- fields
- methods
- forward method declarations
- inherited parent objects

JVMud supports ordinary `inherit` declarations with string paths:

```
inherit "/obj/thing";
```

The parser also recognizes `virtual inherit` syntax for compatibility with mudlib source that uses it:

```
virtual inherit "/obj/base";
```

The runtime resolves object paths through the configured mudlib source root and include paths. Path policy is part of the runtime and mudlib configuration, not the LPC language itself.

Types

JVMud supports the LPC types currently needed by the compiler and hosted mudlibs:

- `int`

- float
- string
- status
- object
- mapping
- mixed
- void
- array forms such as `int *`, `string *`, and `mixed *`

Use explicit return types, field types, local variable types, and parameter types in JVMud-maintained LPC. Explicit types make compiler errors clearer and avoid importing legacy driver ambiguity into the engine contract.

```
string short() {
    return "a brass lantern";
}

int count;

void set_count(int value) {
    count = value;
}
```

Untyped methods and untyped parameters are rejected. Compatibility syntax does not remove the requirement for explicit signatures.

Declarations And Modifiers

Fields and methods may use the common LPC declaration modifiers that JVMud currently recognizes:

- public
- private
- protected
- static
- nomask
- varargs
- nosave
- deprecated

The `varargs` modifier applies to methods only and requires the `varargs` language feature. Protected evaluation, typed function literals, inline callables, and multi-value mappings also have explicit

profile feature gates.

```
private int count;
nosave mixed transient_state;

public string query_name() {
    return "lantern";
}

varargs void message(string text, object target) {
    write(text);
}
```

Multiple field declarators are supported in a single declaration:

```
int x, y, z;
string *names, title_words;
```

Statements

JVMud supports the statement forms used by ordinary mudlib control flow:

- blocks
- expression statements
- if/else
- while
- do/while
- for
- foreach
- break
- continue
- return

Both foreach (`... in ...`) and foreach (`... : ...`) forms are supported. Mappings may be iterated with key and value variables.

```
int total(int *values) {
    int sum;

    foreach (int value in values) {
        sum += value;
    }
}
```

```

    return sum;
}

void describe_mapping(mapping values) {
    foreach (string key, mixed value in values) {
        write(key + ": " + value + "\n");
    }
}

```

Typed local declarations are supported in blocks and in `for` initializers. A `for` initializer may declare one typed local variable.

Expressions

JVMud supports the expression forms expected in practical LPC mudlib code:

- integer, float, string, true, false, and null-like zero values
- array literals
- mapping literals
- local and field access
- assignment and compound assignment
- method calls
- efun calls
- object-style invocation with `→`
- parent calls through `super`
- indexing and slicing
- ternary expressions
- comma expressions in loop clauses
- protected evaluation with `catch`

Supported operator families include arithmetic, comparison, equality, logical, bitwise, shift, and unary operators.

```

mixed *values = ({ "north", "south", "east" });
mapping exits = ([ "north": "/room/green", "south": "/room/road" ]);

string first = values[0];
mixed *rest = values[1..2];

if (sizeof(values) > 0 && exits["north"]) {
    write("There is a way north.\n");
}

```

Adjacent string literals are accepted as one string value, which is useful for long room descriptions:

```
string long() {  
    return "The corridor is narrow and cold. "  
           "A brass lantern hangs from a hook.";  
}
```

Collections

Arrays and mappings are first-class LPC values in JVMud.

Array literals use LPC's (`{ ... }`) form. Mapping literals use LPC's (`[ key: value ]`) form.

```
mixed *items = ({ "lamp", "key", "coin" });  
mapping descriptions = ([  
    "lamp": "A small brass lantern.",  
    "key": "A tarnished iron key."  
]);
```

Indexing is supported for arrays, mappings, and strings where the runtime type allows it. Slicing is supported for strings and array-like values.

Efun Calls

JVMud exposes engine functions as efuncs. Ordinary efun calls use the familiar LPC call form:

```
write("Hello.\n");  
sizeof(({ 1, 2, 3 }));
```

Qualified `efun::name(...)` calls are supported for engine efuncs:

```
return efun::sizeof(({ 1, 2, 3 }));
```

The efun catalog is documented separately in this manual and in the generated Java API documentation for the engine efun registry.

Function References And Inline Callables

JVMud recognizes LPC function-reference forms used by mudlibs, including quoted function references and efun references:

```
#'db_close
```

```
efun::db_handles()
```

Inline callable forms used with collection helpers such as `filter` and `map` are supported for the compatibility patterns currently exercised by hosted mudlibs.

Preprocessor And Includes

JVMud includes a preprocessor with support for ordinary include resolution, macro expansion, conditional directives, and source mapping. Include behavior depends on the runtime/compiler configuration for the mudlib being compiled.

Use includes for shared LPC constants and declarations, but keep engine-facing concepts in JVMud vocabulary. LPC headers should help mudlib code compile; they should not redefine the engine's model of Places, Entities, Players, Sessions, or Personas.

Runtime-Oriented LPC

JVMud supports the LPC patterns needed to host live LPMud objects:

- object loading
- method invocation
- inherited behavior
- selected lifecycle methods
- command hooks such as `init`, `add_action`, and `add_verb`
- player/session text output helpers
- save and restore helpers for LPC object state
- timed behavior through scheduler-backed compatibility functions

These features are compatibility surfaces over JVMud's engine model. They make LPC mudlib code usable without making legacy driver vocabulary the engine's own ontology.

Current Limits

JVMud's LPC support is intentionally practical and evolving. If a historical LPC feature is not needed by the hosted mudlibs, it may be absent, partial, or accepted only as a compatibility parse form.

In particular:

- supported LPC means the JVMud compiler and runtime currently have behavior for the construct, not that every legacy driver variant behaves identically
- compatibility syntax may exist before full runtime behavior exists
- JVMud-authored source should prefer explicit types and simple declarations

- engine concepts should be documented as JVMud concepts, not as legacy LPC driver concepts

When in doubt, prefer the forms already used by JVMud mudlib code and covered by compiler or telnet smoke tests.

Explicit Types

Declare return types and every parameter type. Untyped methods and parameters are compiler errors. Use `mixed` when a value intentionally varies, and `void` when a method returns no value.

These are the default typing rules. A mudlib can explicitly select `compiler.dynamic_types = true` to treat all declared value types as `mixed`. This does not allow missing method declarations; legacy declaration syntax requires its own opt-in. See the compatibility section for the policy boundaries.

For a disposable authoring exercise, save this as `example/counter.c` beneath the selected mudlib source root (create the directory if needed):

```
int count;

int add(int amount) {
    count += amount;
    return count;
}

int increment() {
    return add(2);
}

int query_count() {
    return count;
}
```

In the admin shell, run `load /example/counter`, then use its returned handle with `call <handle> increment` and `call <handle> query_count`. Both calls should return 2 on a fresh object. Use `inspect <handle>` to see its fields and methods. This exercise needs no legacy `efun` aliases. Remove only your disposable example when finished; keep upstream mudlib source intact.

The current admin `call` command passes supplied arguments as strings; it does not coerce numeric text to LPC integers. This exercise uses a no-argument method to call the typed integer helper. Test non-string argument contracts through LPC wrappers or the Java execution API.

Efuncs

The `jvmud_` identifier prefix is reserved for JVMud engine symbols. Mudlib code may call `jvmud_*` `efuns`, but may not declare names with that prefix, including functions, fields, parameters, locals, or macros. This also applies to `mfuns` and compatibility shims. The compiler rejects violations even

when the particular name is not yet an engine `efun`. Strings and comments are unaffected. Profile predefines and engine-function aliases cannot repurpose reserved names either; an alias may retain an engine function's identical native spelling.

Name game policy separately: Small Mercies uses `mercy_random(n)` for favorable dodge rolls and `jvmud_random(n)` for the engine's uniform random primitive. Explicit `efun::` and `jvmud::` calls still bypass mudlib function resolution.

Efuncs expose host operations to LPC. Ordinary calls may resolve through mudlib functions or the active boundary's translation table. `efun::name(...)` selects that `efun` surface; `jvmud::name(...)` requires an exact native JVMud `efun` name.

```
int sample_size() {
    return jvmud::jvmud_size({ 1, 2, 3 });
}
```

The native form does not invent a `jvmud_` prefix or translate `sizeof`. Legacy aliases belong in the profile. A recognized `efun` can still require an `engine_capabilities` grant, such as `mudlib_files`, `database`, `session_control`, or `host_control`.

Use [the `efun` catalog](#) for signatures and categories, and `language.efun.builtin.CoreEfuncs` for implementation details. Do not duplicate the catalog in a mudlib guide. When a call fails, check its spelling, argument types, profile mapping, and required capability.

Ordinary JSON

`jvmud_parse_json(document)` converts an ordinary JSON document to LPC values. JSON objects become mappings with string keys; arrays retain their order; strings and numbers remain scalar values; booleans become status values 1 and 0; and JSON `null` becomes LPC false (0).

```
mapping item = jvmud_read_mudlib_json(
    "/data/items/i0001.json.gz");
```

`jvmud_read_mudlib_json(path)` is available to mudlibs with the `MUDLIB_FILES` engine capability. It parses either plain JSON or gzip-compressed JSON, detected from the file content rather than the filename.

`jvmud_format_json(value)` performs the inverse conversion for values that can be represented in ordinary JSON. Mapping keys must be strings. Because LPC false and a parsed JSON `null` share the value 0, formatting a parsed value is not promised to reproduce the original JSON token-for-token.

Bounded JSON Array Reads

Mudlibs with the `MUDLIB_FILES` engine capability may read a structural slice without materializing an entire JSON document:

```
mixed *items = jvmud_read_mudlib_json_array(
    "/data/catalog.json", "/items", 500, 100);
```

The second argument is an RFC 6901 JSON Pointer selecting an array. `offset` is the zero-based first entry and `count` is the maximum number returned. JVMud scans forward through ordinary JSON, but skips unrelated branches and entries outside the requested range without constructing their LPC values. An empty pointer selects a root array. Plain and gzip-compressed documents are both accepted.

Efun catalog

Boundary Role

JVMud's core efuncs use JVMud-native names. Legacy driver names belong in mudlib compatibility profiles through engine function aliases such as `engine_function.jvmud_size = sizeof`. That keeps familiar LPC source usable without making old driver vocabulary the engine model.

Mudlib code can also qualify calls. `efun::name(...)` bypasses mudlib shim methods while still honoring configured engine-function aliases. `jvmud::name(...)` resolves only JVMud-native efuncs, such as `jvmud::jvmud_size(...)`; it does not invent a `jvmud_` prefix for bad calls.

Current Core Efuncs

This page mirrors the built-in catalog registered by `CoreEfuncs`. Overloaded families are shown with arity ranges when each overload has the same behavior.

Output And Perception

`void jvmud_write (mixed message)`

Writes text to the active output sink.

`void jvmud_write_to_lpc_object (object target, mixed message)`

Writes text to a target object's bound session, when it has one.

`status jvmud_rebind_session_lpc_object (object newObject, object oldObject)`

Moves an interactive session binding from one LPC object to another.

`void jvmud_emit_perceivable (mixed emitter, mixed message)`

Emits a perceivable message from an entity or path-resolved object.

`void jvmud_emit_perceivable_except (mixed emitter, mixed message, mixed excluded)`

Emits near an entity while suppressing one explicit recipient.

`void jvmud_emit_perceivable_at (mixed location, mixed message)`

Emits a perceivable message at an explicit location or path-resolved object.

Current Context

`object jvmud_current_lpc_object ()`

Returns the currently executing LPC object.

`object jvmud_previous_lpc_object ()`

Returns the previous LPC object in the active call chain, when available.

`object jvmud_current_actor ()`

Returns the active command actor, falling back to the current object.

`object jvmud_current_agent ()`

Returns the active world-present Agent for the action scope, or LPC false when none is active.

`string jvmud_current_verb ()`

Returns the verb being dispatched for the current command.

`object jvmud_previous_object ()`

Compatibility spelling for the previous LPC object on the runtime call stack.

Command Dispatch

`mixed jvmud_dispatch_entity_command (mixed actor, string commandLine)`

Dispatches a text command as an entity or path-resolved object.

`void jvmud_enable_commands ()`

Marks the current object as command-capable.

`void jvmud_add_action (string methodName)`

Remembers a method as a command action without registering a verb yet.

`void jvmud_add_action (string methodName, string verb)`

Remembers an action method and registers a verb for it.

`void jvmud_add_action (string methodName, string verb, status prefix)`

Registers a verb for an action method, optionally as a prefix verb.

`void jvmud_add_verb (string verb)`

Registers a verb for the most recently remembered action method in the current interaction scope.

Time And Scheduling

`int jvmud_time ()`

Returns Unix epoch seconds from the host clock.

`string jvmud_format_time (int epochSeconds)`

Formats epoch seconds using the host default time zone.

`int jvmud_random (int max)`

Returns a value in $[0, \text{max})$, or 0 when $\text{max} \leq 0$.

`void jvmud_schedule_recurring_tick (int enabled, int intervalSeconds)`

Schedules or disables recurring temporal ticks for the current object.

`void jvmud_schedule_deferred_callback (string methodName, int delaySeconds)`

Schedules a one-shot callback on the current object.

`void jvmud_schedule_deferred_callback (string methodName, int delaySeconds, mixed arg...)`

Schedules a one-shot callback with one to four mudlib-supplied arguments.

`int jvmud_cancel_deferred_callback (string methodName)`

Cancels matching deferred callbacks and returns the runtime cancellation result.

Objects, Entities, And Containment

`string jvmud_lpc_object_id (mixed object)`

Returns the registered runtime object id for a loaded LPC object.

`status jvmud_method_exists (string name)`

Reports whether the current object exposes a public mudlib method.

`status jvmud_method_exists (string name, mixed object)`

Reports whether another loaded object exposes a public mudlib method.

`array jvmud_inherited_programs (mixed object)`

Returns the transitive LPC program paths inherited by a generated object.

`mixed jvmud_find_lpc_object (string path)`

Returns an already loaded LPC object, or LPC false when it is not loaded.

`mixed jvmud_find_object (string path)`

Returns an already loaded shared object, or LPC false when it is not loaded.

`object jvmud_load_lpc_object (string path)`

Loads or returns the shared LPC runtime object for a mudlib path.

`object jvmud_clone_lpc_object (string path)`

Clones an LPC object and returns the new runtime instance.

`mixed jvmud_invoke_lpc_object (mixed target, string methodName, mixed arg...)`

Invokes an optional method on an object or path-resolved shared object; registered for arities 2 through 6.

`void jvmud_destroy_lpc_object (object object)`

Destroys an LPC object and removes it from runtime indexes.

`void jvmud_set_entity_location (mixed entity, mixed location)`

Sets an entity's location or containment directly.

`void jvmud_move_entity (mixed entity, mixed destination)`

Moves an entity or path-resolved object through runtime containment.

`object jvmud_find_entity (string id)`

Finds an entity by id in the default scope.

`object jvmud_find_entity (mixed id, mixed location)`

Finds an entity by id inside a supplied location or container.

`object jvmud_entity_location ()`

Returns the current object's immediate location.

`object jvmud_entity_location (mixed entity)`

Returns another entity's immediate location.

`object jvmud_first_entity_at (mixed location)`

Starts traversal of entities in a location or container.

`object jvmud_next_entity_at (mixed entity)`

Continues traversal from a previous entity in the same inventory walk.

Aliases, Capabilities, And Light

`void jvmud_bind_entity_alias (object entity, string alias, mixed location)`

Binds a lookup alias for an entity in a location scope.

`object jvmud_find_entity_alias (string alias, mixed location)`

Resolves an alias in a location scope.

`status jvmud_entity_has_alias (mixed entity, string alias)`

Reports whether an entity has a registered alias.

`status jvmud_entity_commands_enabled (mixed entity)`

Reports whether command handling is enabled for an entity.

`void jvmud_set_entity_translucent (mixed entity, status translucent)`

Controls whether ambient perception such as light passes through an entity container.

`status jvmud_entity_translucent (mixed entity)`

Reports whether an entity is currently translucent.

`int jvmud_set_light (int delta)`

Adjusts current runtime light and returns the resulting light value.

Sessions, Users, And Host Requests

`array jvmud_users ()`

Returns the active user/player objects known to the runtime.

`status jvmud_interactive (mixed user)`

Reports whether an object is bound to an active session.

`mixed jvmud_interactive_info (mixed user, int key)`

Returns LPC false for driver-specific interactive metadata JVMud does not expose.

`mixed jvmud_lpc_object_info (mixed object, int key)`

Returns LPC false for driver-specific LPC object metadata JVMud does not expose.

`mixed jvmud_object_info (mixed object, int key)`

Returns LPC false for driver-specific object metadata JVMud does not expose.

`status jvmud_configure_lpc_object (mixed object, int key, mixed value)`

Accepts driver-specific LPC object configuration requests conservatively.

`status jvmud_configure_object (mixed object, int key, mixed value)`

Accepts driver-specific object configuration requests conservatively.

`void jvmud_shutdown ()`

Accepts legacy shutdown requests without exposing driver control to mudlib code.

`void jvmud_set_this_player (mixed player)`

Accepts legacy current-player mutation requests while JVMud keeps session ownership engine-side.

`void jvmud_raise_error (mixed message)`

Raises a runtime exception with the supplied LPC message.

`int jvmud_query_idle (mixed user)`

Returns idle time for a runtime session/user target.

`mixed jvmud_query_ip_number (mixed user)`

Returns the user's remote address, or LPC false when unavailable.

`mixed jvmud_runtime_info (int key)`

Returns LPC false for unsupported runtime metadata queries.

`void jvmud_capture_session_input (string methodName, int noEcho, mixed arg...)`

Routes the next session input line to a method on the current object; registered for arities 2 through 8.

`status jvmud_transfer_player_to_game (string gameId)`

Asks the host to transfer the current player to another registered game.

Files, Persistence, And Databases

`mixed jvmud_read_mudlib_text (string path)`

Reads a mudlib-relative text file, returning LPC false when unavailable.

`mixed jvmud_read_mudlib_text (string path, int startLine, int lineCount)`

Reads a line slice from a mudlib-relative text file for legacy file helpers.

`mixed jvmud_read_mudlib_json (string path)`

Parses a mudlib-relative ordinary JSON file, transparently accepting gzip compression.

`mixed jvmud_read_mudlib_json_array (string path, string pointer, int offset, int count)`

Streams a bounded slice from a JSON array selected by RFC 6901 JSON Pointer, returning LPC false when the file is unavailable.

`array jvmud_list_mudlib_paths (string path)`

Lists mudlib-relative paths with default listing flags.

`array jvmud_list_mudlib_paths (string path, int flags)`

Lists mudlib-relative paths, including simple glob support for compatibility discovery.

`status jvmud_append_mudlib_text (string path, mixed text)`

Appends text to a mudlib-relative file.

`status jvmud_remove_mudlib_text (string path)`

Removes a mudlib-relative file.

`int jvmud_copy_mudlib_text (string source, string destination)`

Copies a mudlib-relative file, returning zero for success.

`int jvmud_rename_mudlib_text (string source, string destination)`

Renames a mudlib-relative path, returning zero for success.

`status jvmud_create_mudlib_directory (string path)`

Creates a mudlib-relative directory.

`status jvmud_remove_mudlib_directory (string path)`

Removes an empty mudlib-relative directory.

`status jvmud_save_lpc_object_state (string path)`

Persists the current object's LPC fields.

`status jvmud_restore_lpc_object_state (string path)`

Restores the current object's LPC fields.

`int jvmud_db_connect (string database)`

Opens a configured JDBC database handle.

`int jvmud_db_connect (string database, string user, string password)`

Opens a JDBC database handle with mudlib-supplied credentials.

`int jvmud_db_exec (int handle, string sql)`

Executes SQL and retains any result cursor for later fetches.

`mixed jvmud_db_fetch (int handle)`

Returns the next result row as an LPC array, or LPC false when no row remains.

`mixed jvmud_db_error (int handle)`

Returns the last SQL error for a handle.

`status jvmud_db_close (int handle)`

Closes a database handle.

`array jvmud_db_handles ()`

Returns currently open database handles.

`string jvmud_db_escape (mixed value)`

Escapes text for mudlib-generated SQL.

Security Helpers

`string jvmud_hash_password (string password)`

Returns a PBKDF2-SHA256 password hash string.

`status jvmud_verify_password (string password, string encodedHash)`

Verifies a password against a hash created by `jvmud_hash_password`.

Text And Regex

`string jvmud_lowercase_text (mixed value)`

Converts text to lowercase.

`string jvmud_uppercase_text (mixed value)`

Converts text to uppercase.

`array jvmud_split_text (string text, string delimiter)`

Splits text on a literal delimiter while preserving empty trailing fields.

`mixed jvmud_regex_match (array values, string pattern)`

Returns values whose string forms match a compatibility regular expression, or LPC false when none match.

`mixed jvmud_regex_match (array values, string pattern, int flags)`

Compatibility regex match with flags.

`string jvmud_regex_replace (string input, string pattern, mixed replacement, int flags)`

Performs compatibility regex replacement for legacy mudlib text helpers, including function callbacks that receive each matched string.

`array jvmud_regex_explode (string input, string pattern)`

Splits text around regular-expression matches while preserving matched delimiters.

`int jvmud_to_int (mixed value)`

Converts numeric values and base-10 text to an integer, returning false-style zero for non-numeric input.

`string jvmud_to_string (mixed value)`

Converts a mixed LPC value to JVMud text form.

`string jvmud_capitalize_text (mixed value)`

Capitalizes the first character of text.

`string jvmud_wrap_text (mixed text)`

Wraps description text at the default 78-column width.

`string jvmud_wrap_text (mixed text, int width)`

Wraps description text at a requested line width.

`string jvmud_format_text (string format, mixed arg...)`

Formats text using the host formatter; registered for arities 1 through 24.

`string jvmud_extract_text (mixed value, int from)`

Extracts text from an inclusive start index through the end.

`string jvmud_extract_text (mixed value, int from, int to)`

Extracts text using inclusive start and end indexes.

Collections, Types, And Values

`int jvmud_size (mixed value)`

Returns the size of a string, collection, mapping, or array.

`int jvmud_member (mixed value, mixed needle)`

Checks mapping key membership or returns the index of a value in an array or string.

`array jvmud_mapping_keys (mapping value)`

Returns a mapping's keys in runtime iteration order.

`array jvmud_mapping_values (mapping value)`

Returns a mapping's values in runtime iteration order.

`mapping jvmud_mapping_from_keys (array keys)`

Builds a mapping whose keys come from an array and whose values are LPC true.

`mapping jvmud_mapping_delete (mapping value, mixed key)`

Removes a key from a mutable mapping and returns the same mapping.

`string jvmud_serialize_lpc_value (mixed value)`

Serializes LPC data values to JVMud's tagged JSON payload.

`mixed jvmud_deserialize_lpc_value (string data)`

Restores LPC data values produced by `jvmud_serialize_lpc_value`.

`mixed jvmud_parse_json (string document)`

Parses ordinary JSON into LPC mappings, arrays, strings, numbers, and 1/0 status values; JSON null becomes LPC false.

`string jvmud_format_json (mixed value)`

Formats LPC-compatible data as ordinary JSON; mapping keys must be strings.

`status jvmud_is_string (mixed value)`

Reports whether the value is Java-backed LPC string data.

`status jvmud_is_int (mixed value)`

Reports whether the value is Java-backed LPC integer data.

`status jvmud_is_object (mixed value)`

Reports whether the value is a live runtime object.

`status jvmud_is_array (mixed value)`

Reports whether the value is Java-backed LPC array data.

`status jvmud_is_mapping (mixed value)`

Reports whether the value is Java-backed LPC mapping data.

`mapping jvmud_filter_indices (mapping values, function callback, mixed arg...)`

Returns entries whose keys satisfy a callable predicate; registered for arities 2 through 8.

`array jvmud_allocate (int size)`

Returns a zero-filled LPC array of non-negative size.

`int jvmud_sscanf (mixed input, mixed format, mixed capture...)`

Scans text into supplied capture targets; registered for arities 3 through 8.

Lifecycle Hooks

JVMud emits native events; a manifest maps them to LPC methods. For example:

```
lifecycle.object_loaded = reset
lifecycle.interaction_scope_started = init
lifecycle.player_session_connected = logon
```

These are illustrative legacy method names, not required names. Native profiles can map the same events to their own typed LPC methods.

Read [the lifecycle reference](#) for each event's role. Verify the mapped method's signature in the profile and corresponding Java dispatch code before adding a hook. Test normal entry, failed input, and disconnect paths; recognizing a method name does not establish its invocation order or error behavior.

Perception

World communication is separate from private session output. An emitted perception reaches the source's location and its immediate contents, excluding the source and any explicitly excluded recipient. It does not recursively reach carried objects or apply sensory rules such as hearing distance or line of sight. Recipients are snapshotted before callbacks; destroyed recipients are skipped.

```
// Emit structured speech with text for connected observers.
jvmud_emit_world_event(this_object(), "speech", "mellon", "A voice says: mellon\n");
// Address one observer without broadcasting to its neighbors.
jvmud_deliver_world_event(target, "speech", "mellon", "A voice whispers: mellon\n");
```

These native functions take (mixed source_or_target, string kind, mixed content, string text). The existing `jvmud_emit_perceivable*` functions emit text events with their original message as content. `jvmud_write_to_lpc_object` remains private output and does not notify world observers.

Map `lifecycle.perception_received` to an observer method taking one mapping. Each observer receives its own mapping containing actor, source, location, kind, content, text, and directed (0 or 1). Missing object references are 0. The originating command actor is preserved during synchronous delivery; events outside a command may have no actor. The receiver is the current object. Text is delivered once to each connected recipient, independently of its hook.

A mudlib may also map `lifecycle.perception_delivery` to a boundary-object method with (object observer, mapping event). It runs after the observer method and can adapt observations to the mudlib's conventions. LP245 uses this to call the optional `catch_tell` method with exact, unwrapped

legacy text. Neither hook is required. Callbacks may emit further events; nesting beyond 64 deliveries raises an error and unwinds the delivery. Callback errors propagate to the existing command/scheduler error handling and can interrupt the remaining recipients.

Lifecycle event reference

How Mudlibs Register Events

A mudlib registers interest through its `MudlibBoundary`. Configuration keys such as `lifecycle.object_loaded = reset` map an engine event to a mudlib method. A mudlib can also declare `handled_lifecycle_events` as metadata before a concrete method mapping exists.

No lifecycle event is globally required. If no method is configured, JVMud continues the engine-owned operation. If a method is configured but the target object does not define it, JVMud skips the call unless a future event contract marks that call required.

Configuration Names

Event names are case-insensitive in mudlib config, and hyphens or spaces normalize to underscores. Older aliases such as `player_connected`, `player_bound`, and `interaction_scope_entered` are accepted for existing manifests, but the current contract names below are preferred.

Current Lifecycle Events

SERVER_STARTED

Implemented. After the declared boundary object and preload manifest have loaded, JVMud invokes the configured no-argument method on the boundary object. A compatibility profile can map a driver-specific master-startup convention here without teaching that convention to the engine.

OBJECT_LOADED

Implemented. Invoked on a loaded or cloned object with one argument, `mixed first_load`; current compatibility passes `0`. Example compatibility mapping: `reset`.

OBJECT_ACTIVATED

Reserved. Names a reactivation moment for reload, reset, or world maintenance, but JVMud does not currently invoke it.

OBJECT_SOURCE_MISSING

Implemented. Invoked on the configured boundary object when a shared object path is requested but no source exists. The method receives the normalized requested path and may return an object to bind to that path. Example compatibility mapping: `compile_object`.

OBJECT_DESTRUCTION_REQUESTED

Implemented. Invoked on the configured boundary object before destruction, passing the target

object. The return value is advisory. Example compatibility mapping: `prepare_destruct`.

OBJECT_DESTROYED

Reserved. Names a post-destruction cleanup point for future policies that need an after-the-fact notification.

ENTITY_ARRIVED_AT_PLACE

Reserved. Names the moment after an entity completes movement into a Place.

ENTITY_DEPARTED_FROM_PLACE

Reserved. Names the moment when an entity leaves a Place, paired with `ENTITY_ARRIVED_AT_PLACE`.

ENTITY_ADDED_TO_ENTITY

Reserved. Names the moment when an entity enters another entity's containment.

ENTITY_REMOVED_FROM_ENTITY

Reserved. Names the moment when an entity leaves another entity's containment.

PLAYER_SESSION_CONNECTED

Implemented by the hosted server path. The configured player object method is invoked without arguments and may start login or character selection. Example compatibility mapping: `logon`.

PLAYER_SESSION_POST_REBIND

Implemented by the hosted server path after a compatibility session handoff changes which LPC object receives player input. The configured method is invoked without arguments on the newly bound object.

PLAYER_PERSONA_RESOLVED

Implemented for host-managed session policies. Delivers the external user ID, display name, gender text, and policy-defined profile attributes to the selected Persona.

PLAYER_OBJECT_BOUND

Reserved. Names the moment when JVMud has associated a live mudlib projection with a Player, Session, and Persona.

PLAYER_ENTERED_WORLD

Implemented for host-managed session policies. Delivers the configured starting Place object after the Player's Persona has entered the world.

PLAYER_SESSION_DISCONNECTED

Implemented by the hosted server path. The configured player object method is invoked without arguments so the mudlib can save or clean up session-owned state. Example compatibility mapping: `quit`.

INTERACTION_SCOPE_STARTED

Implemented. Invoked without arguments on the active Persona, its location, objects in that location, and carried objects. Compatibility mudlibs usually register local text commands here.

Example compatibility mapping: `init`.

COMMAND_DISPATCH_STARTED

Reserved. Names the moment before JVMud dispatches a Player command line to mudlib behavior.

COMMAND_DISPATCH_FINISHED

Reserved. Names the moment after JVMud finishes dispatching a Player command line.

LOG_ERROR

Implemented. Invoked on the configured boundary object with the object path and diagnostic message for compilation or load-time diagnostics. Example compatibility mapping: `log_error`.

RUNTIME_ERROR

Implemented for timed runtime errors and server-layer error handling. Invoked on the configured boundary or error handler object with details about target, context, operation, and message. Example compatibility mapping: `runtime_error`.

SCHEDULED_TICK_ERROR

Implemented. When a scheduled tick throws, JVMud can invoke this specialized error event in addition to `RUNTIME_ERROR`. Example compatibility mapping: `heart_beat_error`.

SERVER_SHUTDOWN

Implemented by the hosted server path. Invoked on the configured boundary or error handler object with the shutdown reason. Example compatibility mapping: `notify_shutdown`.

SCHEDULED_TICK

Implemented through temporal tick configuration rather than a per-event method mapping. The boundary chooses the recurring tick method and default interval; compatibility shims may expose names such as `set_heart_beat`.

DEFERRED_CALLBACK

Reserved as a named event. Concrete one-shot callbacks are currently scheduled with explicit callback method names through runtime `efuns`.

Compatibility Shims

Keep imported source as upstream material. Put compatibility functions, aliases, headers, and lifecycle translations in the profile's `jvmud/` area where possible. `mfun_object` supplies mudlib-global helpers; `compatibility_object` and `compatibility_overrides` support explicit compatibility overrides.

For a missing legacy operation, first identify whether the gap is syntax, a native engine operation, or profile policy. Add generic compiler/engine behavior only when it belongs to JVMud; map legacy vocabulary in the profile. Do not relax explicit typing to accommodate an untyped legacy declaration.

Record the motivating source and test in `docs/COMPATIBILITY_FEATURE_PROVENANCE.md` when changing

compatibility features. The result should remain usable without hardcoding a particular game into shared Java or launch scripts.

Opt-In Legacy Method Transpilation

A bridge manifest may explicitly set `transpilation.untyped_methods = true`. The default is false. JVMud's `io.github.protasm.jvmud.language.transpiler` subpackage adds explicit mixed return and parameter types to untyped method signatures in memory after preprocessing and before parsing. Includes, inherited objects, and global helper objects use the same profile setting. Existing explicit types and method bodies remain unchanged, and inserted tokens retain the original source locations. The compiler still rejects missing explicit method types when transpilation is disabled. This accommodation does not imply support for every legacy LPC dialect feature and never rewrites imported files.

Opt-In Dynamic Value Typing

Set `compiler.dynamic_types = true` when a mudlib's value declarations are documentary. The default is false. The compiler retains written type names but uses stable mixed storage for fields, locals, parameters, return values, and typed function literal declarations. Array declarations impose no restriction. Includes, inheritance, global helpers, and hosted bridge objects use the same policy. Assignments cannot narrow storage after compilation.

Modifiers, argument structure, native function contracts, explicit conversions, and runtime operation checks remain in effect. Duplicate methods of the same arity remain errors. Missing declarations require the separate untyped-method switch. Mixed defaults and empty returns use LPC zero. Existing unsupported syntax, including legacy casts, remains unsupported.

LP245 selects this policy instead of its former 51 field and 24 local type overrides. Its JSON file retains only the method-varargs rule for `room/room.c:init`. Remove field/local type overrides before enabling dynamic typing; combining them is rejected. Checked overrides remain available for mudlibs using strict typing.

Restart the instance when changing typing policy so every object is recompiled under the same policy.

Checked Field-Type Overrides

Select a bridge-owned JSON file explicitly in the manifest:

```
transpilation.overrides = transpilation.json
```

The override file path is relative to the manifest directory. This setting is independent of `transpilation.untyped_methods`; neither enables the other.

```
{
```

```

"field_type_overrides": [
  {
    "file": "obj/torch.c",
    "field": "amount_of_fuel",
    "expected_type": "string",
    "replacement_type": "int"
  }
]
}

```

file identifies a filesystem compilation unit relative to `mudlib_root`. Rules apply after preprocessing, so declarations expanded from its headers are part of that unit. They also apply when the unit is loaded as an inherited object, global helper, or bridge object. In-memory snippets without a source file path are not matched by file overrides.

The named field must occur exactly once with the expected type. A mismatch, missing field, or duplicate declaration stops compilation at the TRANSPILE stage. Invalid JSON, unknown keys, duplicate rules, missing source files, and unsupported field types are errors. Fields can use `int`, `float`, `string`, `object`, `mixed`, `mapping`, `function`, or `status`, with optional array stars. The JSON vocabulary also supports `local_type_overrides` and `method_varargs_overrides`; those adaptations are separate from field rules.

Grouped declarations are split when necessary to preserve neighboring fields' types, modifiers, and initializer order. Locals, parameters, method bodies, and string values are not field override targets. Source locations are retained for diagnostics and original files are never rewritten. Rules are read at manifest load; restart the instance to load changed configuration.

Mudlibs

A profile combines authored LPC with explicit hosting requirements. Native and legacy profiles share the JVMud host while owning their own vocabulary and game rules.

Layout

A useful source layout separates authored content, host configuration, and writable state. This is an illustrative convention, not a required game:

```

<content-root>/
  source/           LPC source, headers, and authored data
  jvmud/            manifest and host-facing adapters
  <state-area>/     profile-selected writable data

```

`mudlib_root` selects the actual source root. If the manifest lives in `jvmud/`, `mudlib_root = ../source` resolves from that directory. Inspect persistence code before choosing, moving, or cleaning state paths; not every store uses a sibling state directory.

Supply peer manifests to `scripts/jvmud-start <manifest> <manifest> ...`. The engine offers them in a connection menu. Each must have a unique game id. `mount.*` declarations and world-hopping are unsupported. No manifest owns or implicitly discovers other worlds.

Small Mercies follows this separation under `mudlibs/smallmercies/`. Its guest characters are transient; it does not demonstrate durable account storage.

Validate a profile

Before hosting content, check these responsibilities against your source tree:

- the manifest's initial Place and Persona paths resolve to real LPC objects;
- includes, inherited objects, and preloads resolve under the intended root;
- lifecycle mappings name methods with the expected signatures;
- optional language features and engine capabilities match actual requirements;
- writable stores exist where the persistence policy expects them;
- each configured world boots independently and can be selected directly from the engine menu.

For a load failure, reproduce the same object under the same manifest in the admin shell with `verbosity watch`. Changes to includes, syntax flags, or adapters can affect many objects, so rerun focused tests and the relevant live route.

Upstream Source Policy

Preserve upstream material and provenance. Prefer dedicated shims and manifest mappings to broad source rewrites. Keep necessary source fixes small and tested; a content bug is not automatically a compiler defect. Never commit private saves or secrets. Use `database.password_env` for database credentials.

Developer Reference

Use the core package map to choose where a change belongs, then follow the compiler and hosted-instance flows below. The [Java API](#) provides class and method contracts for all six core package families.

Compiler Pipeline

JVMud is easiest to understand as a collaboration between compiler, engine, and mudlib. The compiler turns LPC source into JVM classes. The engine owns the live world model. The mudlib supplies authored behavior and compatibility policy.

The compiler pipeline is deliberately staged so each step has a small job and a clear artifact:

Stage	Input	Output
Preprocess	LPC source text plus include paths and macro definitions	Expanded source with mappings back to the original files
Scan	Preprocessed source	A token stream
Parse	Tokens	An abstract syntax tree for one LPC object
Analyze	AST plus inherited object metadata and registered efun signatures	A semantic model with resolved names, signatures, and type information
Lower	Semantic model	Typed intermediate representation
Compile	Typed IR	JVM bytecode
Execute	Bytecode plus host runtime configuration	A loaded LPC object instance managed by the host-facing LPC runtime

The package layout follows that path. `preproc`, `scanner`, `token`, `parser`, `semantic`, `ir`, and `bytecode` are compiler stages. `pipeline` coordinates the stages and reports `CompilationProblem` values when a source unit cannot move forward. `efun` defines the LPC-facing function contract, and `efun.builtin.CoreEfuncs` provides JVMud's built-in catalog.

The package `io.github.protasm.jvmud.language.runtime` is support code for compiled LPC bytecode: active execution context, truthiness, coercions, indexing, strings, arrays, and runtime type descriptors. It is not the JVMud engine ontology. Engine-owned concepts live under `io.github.protasm.jvmud.execution.application` with `execution.model.world` for `WorldRuntime`, `Place`, and `Entity`; `execution.model.identity` for `Player`, `Session`, and `Persona` records; `execution.model.time` for scheduling; and `execution.model.mudlib` for `mudlib` lifecycle boundaries.

Instance And Transport Internals

`execution.application.JVMud` starts engine endpoints independently of `mudlibs` and supervises one sandboxed worker JVM per `mudlib`. `MudlibWorker` boots through `MudlibBoot` and reports readiness over private pipes. The engine then publishes the `mudlib`'s player/admin pair. `PlayerGateway` provides the engine menu and direct relays; worker-side `TelnetServer` handles only its own `mudlib`'s protocol sessions. There is no return-to-menu operation after a player selects a `mudlib`.

Each `MudInstance` owns a `MudlibExecution`: one thread serving its command queue and clock. Player input, protocol callbacks, administration, and scheduled work run serially there. Each manifest supplies its own `temporal_tick_interval`; zero disables automatic ticks. Ticks use a delay after the preceding tick completes, without accumulating catch-up ticks. No router-wide tick loop or cross-world execution lock exists.

Telnet delivers input without interpreting LPC command results or generating rejected-command messages. The selected `mudlib` owns that meaning and output.

Follow an input from transport to instance to LPC command handling when debugging player behavior. Follow an efun from its catalog through runtime context and instance services when debugging host effects. `language.runtime` is generated-code support; `execution.model.world` owns the engine's world model.

`communication.console.AdminConsole` connects to an engine or mudlib administration endpoint. The engine owns TLS authentication and grants. Engine commands execute in `EngineAdminCommandSession`; mudlib commands cross private worker pipes and execute in `MudlibAdminCommandSession` on that instance's execution queue. No administrator credentials are sent to worker JVMs. Player listeners relay bytes to a selected worker, where Telnet negotiation, player login and game commands are handled.

Storage adapters live under `storage`; engine administrator identities are separate from mudlib player accounts. See `docs/ENGINE-ADMINISTRATION.md` for the OS sandbox boundary, supervision deadlines and platform limitations.

Testing And Verification

Run the repository baseline from its root:

```
mvn test
```

For a targeted change, select the relevant JUnit class, for example:

```
mvn -Dtest=MudlibAdminCommandSessionTest,EngineArchitectureTest test
```

Some integration checks require an external database and configured password environment variable. A missing prerequisite is an incomplete integration check, not a passing compatibility result. Read the failing test and profile before classifying it as a Java regression.

Use `target/surefire-reports/` for test outcomes. Informational compatibility reports and full preload scans answer different questions from assertions and live player tests. Shared scan tooling is `scripts/scan-mudlib-preload-compile.sh`; profile wrappers remain under the owning mudlib's `jvmud/` directory.

For runtime changes, restart the affected profile and replay the relevant player route. For documentation-only edits, regenerate outputs and check local links, anchors, commands against source, and PDF layout. See [Documentation Generation](#).

Troubleshooting

Begin with the smallest failing step: Java build, manifest validation, object load, listener startup, login, or command handling. Keep the first diagnostic and the exact manifest path; later errors may be consequences of the first failure.

Build Failures

- **Release 21 not supported:** compare `java -version` and `mvn -version`; Maven may be using a different JDK. Select the intended JDK before rebuilding.
- **Dependencies cannot resolve:** check access to the configured Maven repository. An offline cache must already contain the required artifacts.
- **Tests fail on database setup:** inspect the selected profile's external service requirements. Do not put a password in the manifest to silence the failure.
- **Permission denied launching a script:** use a POSIX shell environment and check the script's executable bit in the checkout.

Retain Maven's first error and the relevant Surefire report. Repeatedly rebuilding without changing the failing prerequisite is unlikely to help.

Object Load Failures

1. Confirm the source path is relative to the selected mudlib root, not the Java repository root.
2. Inspect `include_paths`, inherited source paths, and preload declarations.
3. In the admin shell, set `verbosity watch` and run `load <path>`.
4. Use `preprocess`, `scan`, `parse`, or `ir` to stop at the earliest failing stage.
5. Check explicit types, optional `language_features`, `efun` aliases, and capability grants before changing source semantics.

Unknown manifest keys fail fast. A missing `initial_place` or invalid profile mapping is a hosting configuration problem. Runtime invocation failures after a successful load need an execution test, not just another compile scan.

Login And Command Failures

Symptom	Check
Connection refused	Wait for the listening message; verify the process is still running on the configured address and player port (default <code>localhost:4000</code>).
Address already in use	Stop your earlier listener or choose another pair with <code>--port 4010 --admin-port 4011</code> .
Prompts repeat or login fails	Confirm the selected profile and account policy; inspect the server log and that profile's writable storage.
Admin command is unrecognized in Telnet	Use <code>scripts/jvmud-console --port <admin-port></code> ; the server must have been started with <code>--admin-port</code> .

Symptom	Check
Movement or look is unrecognized	Complete login and read the active mudlib's help; verify command registration and lifecycle mapping.
GMCP output is absent	Check client negotiation, <code>session_control</code> , and the mudlib's package emission. GMCP is not required for ordinary text play.

For mounted-world issues, first prove each profile can boot directly, then test the engine menu selection and the selected mudlib manifest. Capture the first failing transition.

Documentation Generation

Edit `manual/*.adoc`, not `docs/manual/index.html` or the PDF. From the repository root, generate both manual formats:

```
mvn -Pmanual-docs -DskipTests process-resources
```

The outputs are `docs/manual/index.html` and `docs/manual/jvmud-user-manual.pdf`. Shared HTML styling and navigation come from `manual/docinfo.html` and `manual/docinfo-header.html`; PDF typography and page flow come from `manual/pdf-theme.yml`.

Generate the Java API with:

```
python3 scripts/build-api-docs.py
python3 src/test/scripts/check-docs.py
```

Javadocs live under `docs/api/apidocs/`; `docs/api/index.html` is the generated package directory. The build script runs Javadoc, derives package descriptions from its output, and opens detailed reference links in a new tab. Standard Javadoc pages retain their own layout. The checker verifies local HTML links, fragment targets, and manual placeholders. It does not check external URLs or prove visual quality.

Preview the static site with `python3 -m http.server 8000 --directory docs` and open <http://localhost:8000/>. Stop that preview with Ctrl+C.

If generation fails, check Maven dependency resolution, missing AsciiDoc includes, and the first generator error. Regenerate HTML and PDF together, then inspect the PDF contents, page breaks, and long code/table cells before publishing. See `docs/README.md` for source ownership and `docs/DOCUMENTATION_GAPS.md` for remaining coverage work.

Appendixes

Use these references while following the procedures. The manifest key inventory and command syntax describe the current implementation; compatibility limits are explicit rather than implied by a successful build.

Commands

These mudlib commands run after connecting the administration console to a mudlib-admin endpoint. Engine endpoints have their own `help` command. Paths name source files; handles identify loaded objects. Use `objects` to find handles. Arguments to `call` are passed as strings; quote strings containing spaces. Numeric text is not automatically converted to LPC integers. `help` is the live syntax reference.

Command	Alias	Purpose
<code>help</code>	<code>h</code>	Show this command reference.
<code>call <handle> <method> [args...]</code>	—	Invoke a method on a loaded object handle.
<code>cat <path></code>	—	Print a file from the virtual mudlib filesystem.
<code>cd [path]</code>	—	Change the current virtual mudlib directory.
<code>clone <path></code>	—	Compile if needed and create a new LPC object instance.
<code>compile <path></code>	<code>c</code>	Compile an LPC source file to bytecode without loading it.
<code>destruct <handle></code>	<code>x</code>	Remove an object from the runtime.
<code>inspect <handle></code>	<code>i</code>	Show object state, inventory, environment, and methods.
<code>ir <path></code>	<code>ir</code>	Compile through lowering and print the typed IR.
<code>load <path></code>	<code>l</code>	Compile, load, and register an LPC object.
<code>look <handle></code>	<code>k</code>	Call <code>long()</code> or <code>short()</code> and display object text.
<code>ls [path]</code>	—	List a virtual mudlib directory or file.
<code>move <handle> <dest></code>	<code>m</code>	Move one object handle into another object's inventory.
<code>objects</code>	<code>o</code>	List registered live object handles.
<code>parse <path></code>	<code>p</code>	Preprocess, scan, parse, and print the LPC AST.
<code>preprocess <path></code>	<code>pp</code>	Expand includes, macros, and preprocessor directives.
<code>pwd</code>	—	Print the current virtual mudlib directory.
<code>reload <path></code>	<code>r</code>	Recompile and replace a loaded LPC object.

Command	Alias	Purpose
scan <path>	s	Preprocess and print scanner tokens for an LPC source file.
verbosity [quiet normal watch]	v	Show or change compiler/shell output detail.
where <handle>	w	Show the environment containing an object.
quit	q	Disconnect this console session; the mudlib keeps running.

The terminal launcher for play is `scripts/jvmud-start [--bind <address>] [--port <port>] [--admin-port <port>] [--state-dir <path>] [--trace-startup-loads] [<mudlib-config-file>...]`. Inside a player client, profiles using the default transport prefix reserve `//help` and `//quit` for transport controls. The mudlib owns all gameplay verbs.

Configuration Reference

Manifests use `key = value`, one declaration per line. Lists are comma-separated. Blank lines and comments are ignored. Simple surrounding quotes are removed; this is not a general-purpose quoted-string format, and still starts a comment. Unknown keys are rejected by `MudlibBoundaryConfigReader`.

Use a checked-in profile as the complete example. The following groups cover the reader's exact keys and supported key prefixes. Optional values can still become required by the selected boot, session, or database policy.

Identity and paths

Key	Meaning
game_id, game_name	Stable profile identity and display name.
mudlib_root	LPC source root; relative paths resolve from the manifest directory.
initial_place	Startup Place object path, required for hosted boot.
persona_object, player_object	Interactive object path; <code>persona_object</code> takes precedence over the legacy spelling when both are present.
mudlib_object	Optional boundary-declaration LPC object.
mfun_object	Mudlib-global function object.
include_paths	Comma-separated source include search paths.
preload_file	Startup object manifest, relative to <code>mudlib_root</code> (for example, <code>preload_objects</code>). List one object path per line in load order; blank lines and lines beginning with <code>#</code> are ignored. Object paths are relative to <code>mudlib_root</code> , with an optional <code>.c</code> extension. Prefer this manifest for the startup list.

Key	Meaning
<code>preload_objects</code>	Additional comma-separated startup object paths.

Use mudlib-rooted object paths such as `room/church`; Java package paths are not LPC paths. The launch scripts resolve a relative manifest argument from the repository root. `mudlib_root` resolves from the manifest, a different base.

Sessions and presentation

Key	Meaning
<code>session_policy</code>	Optional host-managed login policy. <code>filesystem_accounts</code> selects reusable local account persistence; omit for profile-owned login.
<code>player_prompt</code> , <code>command_prompt</code>	Prompt text; <code>player_prompt</code> takes precedence.
<code>presentation.connected_banner</code>	Text shown on connection.
<code>transport.control_prefix</code>	Escape prefix for transport controls; default <code>//</code> .
<code>diagnostics.location_command</code>	Optional profile-selected location diagnostic command.
<code>max_line_length</code>	Presentation line width, validated against engine bounds.
<code>show_ruler</code>	Presentation ruler toggle. Booleans accept <code>true/false</code> , <code>yes/no</code> , <code>on/off</code> , or <code>1/0</code> .

Host and port are not manifest keys. The shared Telnet launcher uses loopback port 4000; passing a port as a second positional argument fails.

Language and host access

Key	Meaning
<code>language_features</code>	Optional syntax gates: <code>protected_evaluation</code> , <code>typed_function_literals</code> , <code>inline_callables</code> , <code>multi_value_mappings</code> , <code>varargs</code> .
<code>compiler.dynamic_types</code>	Optional dynamic value typing, default <code>false</code> . Treat declared fields, locals, parameters, and returns as stable mixed values. Cannot be combined with field/local type overrides.
<code>transpilation.untyped_methods</code>	Supply missing method return and parameter types in memory; default <code>false</code> and independent of dynamic typing.
<code>transpilation.implicit_self_calls</code>	Resolve unknown bare calls on the current object at runtime; default <code>false</code> .
<code>transpilation.overrides</code>	Manifest-relative JSON file containing checked field, local, or method-varargs adaptations. LP245 retains only a method-varargs rule.

Key	Meaning
<code>engine_capabilities</code>	Capability grants: <code>mudlib_files</code> , <code>database</code> , <code>session_control</code> , <code>host_control</code> .
<code>engine_function.<native-name></code>	One or more mudlib aliases for a native engine function.
<code>compatibility_object</code>	Explicit compatibility-global function object.
<code>compatibility_overrides</code>	Names allowed to override mudlib-global functions.
<code>compatibility.predefine.<macro></code>	Object-like compatibility macro replacement.
<code>compatibility.function_predefine.<macro>.<argument></code>	Single-argument function-like compatibility macro replacement.

Feature flags enable syntax; capabilities enable restricted host operations. Start with the active profile's actual requirements. A compatibility alias does not bypass a capability grant.

Lifecycle and time

Key	Meaning
<code>handled_lifecycle_events</code>	Comma-separated native event declarations.
<code>lifecycle.<event></code>	Map a native lifecycle event to a mudlib method. Event names are normalized for case, spaces, and hyphens.
<code>temporal_tick_method</code>	Mudlib method for scheduler-backed recurring behavior.
<code>temporal_tick_interval</code>	Interval in seconds by default; also accepts <code>s</code> , <code>seconds</code> , <code>ms</code> , or <code>milliseconds</code> . Negative values fail. Zero does not install an interval override.

Refer to [lifecycle events](#) for event roles and verify method signatures against the selected profile.

Database

Key	Meaning
<code>database.url</code> , <code>database.jdbc_url</code>	JDBC URL; <code>database.url</code> takes precedence.
<code>database.user</code>	Database user for the selected profile.
<code>database.password_env</code>	Name of a required nonblank environment variable containing the password. Preferred for configured profiles.
<code>database.password</code>	Literal password alternative supported by the reader; do not commit secrets. Mutually exclusive with <code>database.password_env</code> .

A missing named password environment variable fails manifest loading. Setting the variable does not provision a database, create a schema, or grant database access. Those are profile-specific prerequisites.

Compatibility Notes

LPC dialects differ. Accepted syntax, successful compilation, object loading, and correct interactive behavior are separate levels of support. Optional language features are selected per profile; legacy efun names and lifecycle methods are boundary mappings.

`docs/COMPATIBILITY_FEATURE_PROVENANCE.md` records why compatibility features exist.

`docs/DEFERRED_WORK.md` records implementation debt; `docs/FUTURE_DESIGN_IDEAS.md` collects proposals. None is a substitute for the active source and tests, or a guarantee of full legacy-driver parity.

Database provisioning and recovery remain profile-specific. Start with the credential-free examples in this manual before taking on that environment.

Glossary

Core Concepts

Game

The playable system JVMud hosts: rules, mechanics, goals, challenges, progression, or other game-like structures.

Text

The primary medium through which the world is perceived and manipulated.

Interactive

The property that players can affect the world and the world can affect players.

Multiplayer

The property that multiple players inhabit the same world at the same time.

World

The complete shared virtual domain of the game, composed of places, links, entities, locations, containment, and movement.

Persistence

The property that the world and its state endure independently of any individual session.

Temporality

The property that the world can change through time.

Presence

The requirement that a player experiences the game from within the world through a located persona.

World Model

Place

A discrete location in game space where entities may exist, be perceived, or interact.

Link

A traversable relationship between places. Links make the world navigable.

Entity

A thing in the world: a player persona, coin, sword, bag, vehicle, NPC, effect, or environmental feature.

Location

The mechanical relationship that says where an entity is. A location is either a place or another entity.

Containment

The relationship in which one entity is inside, carried by, mounted on, or otherwise immediately located in another entity.

Single Containment

The rule that every entity has exactly one immediate location.

Movement

The transition of an entity from one location to another, often through a place-to-place link.

Player And Connection Model

Player

The human-facing controller endpoint associated with a session. JVMud does not make Player an account, login, or durable identity.

Session

An active connection or control context, such as a Telnet connection.

Persona

The entity currently associated with a session as that session's in-world perspective.

Player Object

Compatibility vocabulary for an LPC-authored mudlib object that may combine profile data, persona behavior, login flow, and legacy session glue.

Interaction Scope

The local command and perception context around a persona.

Command Dispatch

The engine process that routes one line of player input to mudlib behavior registered for the

current interaction scope.

LPC And Compatibility

LPC

The language used by JVMud's sole mudlib target.

LPMud

A MUD whose game world is authored in LPC, compiled into live game objects, and reloadable while the game continues running.

Mudlib

The LPC source and content layer that defines world fiction, object behavior, commands, rules, and presentation.

Compatibility Shim

A mudlib-side object dedicated to translating legacy LPC/LPMud vocabulary into JVMud-native engine requests.

Mfun

A mudlib-global function provided by a configured mudlib object.

Efun

An engine function callable from compiled LPC code.

Lifecycle Hook

A configured engine-to-mudlib call for a JVMud lifecycle event.

Live Reload

The requirement that LPC-authored objects can be rewritten, recompiled, and reloaded while the game continues running.

Legacy Terms

Room

A legacy mudlib term for a location-like object. JVMud's engine concept is place.

Heartbeat

A legacy driver concept for recurring object callbacks. JVMud's engine concept is temporality and scheduled time.

Call Out

A legacy driver concept for delayed callbacks. JVMud treats delayed work as a mudlib request to the engine's temporality model.

Apply

A legacy driver term for driver-invoked mudlib methods. JVMud uses explicit lifecycle hooks.

Master Object

A legacy driver policy object. JVMud uses explicit boundary configuration and compatibility shims.